Build: Shopping Cart with Context

3 min read Updated 23 Apr 2026 Topic 62 of 62

Bookmark Mark done

Problem: Build a shopping cart with add, remove, update quantity, and total calculation using Context API.

What This Tests: Context + useReducer, complex state transitions, derived values (total), component communication.

jsx

JSX Copy

```jsx
const CartContext = createContext();

function cartReducer(state, action) {
  switch (action.type) {
    case 'ADD_ITEM': {
      const existing = state.find(item => item.id === action.product.id);
      if (existing) {
        return state.map(item => item.id === action.product.id ? { ...item
      }
      return [...state, { ...action.product, quantity: 1 }];
    }
    case 'REMOVE_ITEM':
      return state.filter(item => item.id !== action.id);
    case 'UPDATE_QUANTITY':
      if (action.quantity <= 0) return state.filter(item => item.id !== ac
      return state.map(item => item.id === action.id ? { ...item, quantity
    case 'CLEAR':
      return [];
    default:
      return state;
  }
}
```

```
  const total = cart.reduce((sum, item) => sum + item.price * item.quantit
  const itemCount = cart.reduce((sum, item) => sum + item.quantity, 0);

  return (
    <CartContext.Provider value={{ cart, dispatch, total, itemCount }}>
      {children}
    </CartContext.Provider>
  );
}

function useCart() {
  const context = useContext(CartContext);
  if (!context) throw new Error('useCart must be used inside CartProvider'
  return context;
}

// Product list
function ProductList() {
  const products = [
    { id: 1, name: 'React T-Shirt', price: 25 },
    { id: 2, name: 'JavaScript Mug', price: 15 },
    { id: 3, name: 'Node.js Sticker', price: 5 },
  ];
  const { dispatch } = useCart();

  return (
    <div>
      <h2>Products</h2>
      {products.map(product => (
        <div key={product.id} style={{ display: 'flex', justifyContent: 's
          <span>{product.name} - ${product.price}</span>
          <button onClick={() => dispatch({ type: 'ADD_ITEM', product })}>
        </div>
      ))}
    </div>
  );
}

// Cart display
function Cart() {
  const { cart, dispatch, total, itemCount } = useCart();
```

```
  return (
    <div>
      <h2>Cart ({itemCount} items)</h2>
      {cart.map(item => (
        <div key={item.id} style={{ display: 'flex', justifyContent: 'spac
          <span>{item.name} - ${item.price}</span>
          <div>
            <button onClick={() => dispatch({ type: 'UPDATE_QUANTITY', id:
            <span style={{ margin: '0 8px' }}>{item.quantity}</span>
            <button onClick={() => dispatch({ type: 'UPDATE_QUANTITY', id:
            <button onClick={() => dispatch({ type: 'REMOVE_ITEM', id: ite
          </div>
        </div>
      ))}
      <h3>Total: ${total.toFixed(2)}</h3>
      <button onClick={() => dispatch({ type: 'CLEAR' })}>Clear Cart</butt
    </div>
  );
}

// App
function App() {
  return (
    <CartProvider>
      <ProductList />
      <Cart />
    </CartProvider>
  );
}
```

Key Concepts: Context + useReducer for global state, add-or-increment logic (checking if item exists), derived values (total, itemCount), removing items when quantity reaches 0, custom useCart hook with safety check, Provider pattern, component separation (ProductList and Cart are independent).

QUICK CHECK — no marks, just for you.

1. ... happens when ADD_ITEM is dispatched with a product whose id is already in the cart?

- The existing entry is removed and replaced with a fresh entry of quantity 1
- The action is ignored and the previous state is returned unchanged
- A second entry for the same product is appended with quantity 1
- The matching item's quantity is incremented by 1 and no new entry is appended

Check answer

2. The minus button dispatches UPDATE_QUANTITY with quantity: item.quantity - 1. If an item currently has quantity 1 and the user clicks minus, what does the reducer return?

- A cart with that item filtered out entirely
- A cart where that item now has quantity 0 but is still listed
- A cart unchanged, because negative and zero quantities hit the default case
- A cart where the quantity is clamped back up to 1

Check answer

3. According to the lesson's design, how are total and itemCount produced in CartProvider?

- They are stored on the Context object itself and mutated directly by dispatch
- They are held in separate useState calls that each action handler must keep in sync
- They are derived from the cart array on each render, not stored in reducer state
- They are extra fields inside the reducer's state object, updated by every action

Previous
Build: Pagination Component

LEARN MORE

ARTICLES & DOCS

Scaling Up with Reducer and Context
react.dev

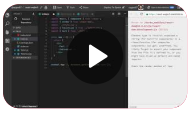
VIDEOS



Shopping cart with Context + useReducer
YouTube



Shopping cart (Context API + useReducer)
YouTube



Context API shopping cart (hooks)
YouTube

Hand-picked references for this lesson.